Week 9: Sales and Customer Analytics System

2. Generate Total Sales Reports

Tasks:

Write SQL queries to calculate:

1. Total number of orders placed.

```
SQL> SELECT COUNT(*) AS NUMBER_OF_ORDERS_PLACED FROM ORDERS;

NUMBER_OF_ORDERS_PLACED
-----------------------
                      5
```

2. Total revenue generated from sales.

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS;

TOTAL_REVENUE
-------------
         4070
```

3. Average order value.

```
SQL> SELECT ROUND(AVG(TOTAL_AMOUNT) ,2) AS AVEREAGE_ORDER_VALUE FROM ORDERS;

AVEREAGE_ORDER_VALUE
--------------------
                 814
```

4. Highest and lowest order amount.

```
SQL> SELECT MAX(TOTAL_AMOUNT) AS HIGHEST_ORDER, MIN(TOTAL_AMOUNT) AS LOWEST_ORDER FROM ORDERS;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
         1250          480
```

5. Total sales for a specific period

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE FROM ORDERS WHERE ORDER_DATE BETWEEN DATE '2026-05-10' AND DATE '2026-09-23';

TOTAL_REVENUE
-------------
         2690
```

3. Customer Purchase Analysis

1. Total number of orders placed by each customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS TOTAL_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C
.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_ORDERS DESC;

                    CUSTOMER_ID CUSTOMER_NAME          TOTAL_ORDERS
--------------------------------- -------------------- ------------
                            105 KAMESH                          1
                            102 VIJAY KUMAR                     1
                            101 SNEHA                           1
                            103 ANITHA                          1
                            104 SURESH                          1
```

2. Total amount spent by each customer

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, NVL(SUM(O.TOTAL_AMOUNT) ,0) AS TOTAL_AMOUNT_SPENT FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.C
ER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_AMOUNT_SPENT DESC;

                    CUSTOMER_ID CUSTOMER_NAME          TOTAL_AMOUNT_SPENT
--------------------------------- -------------------- ------------------
                            102 VIJAY KUMAR                        1250
                            105 KAMESH                              900
                            104 SURESH                              890
                            101 SNEHA                               550
                            103 ANITHA                              480
```

3. Average spending per customer.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, ROUND(NVL(AVG(O.TOTAL_AMOUNT) ,0), 2) AS AVERAGE_SPENDING FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID =
O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY AVERAGE_SPENDING DESC;

                    CUSTOMER_ID CUSTOMER_NAME          AVERAGE_SPENDING
--------------------------------- -------------------- ----------------
                            102 VIJAY KUMAR                        1250
                            105 KAMESH                              900
                            104 SURESH                              890
                            101 SNEHA                               550
                            103 ANITHA                              480
```

4. Customers with maximum purchase amount.

```
SQL> SELECT C.CUSTOMER_ID,
  2         C.CUSTOMER_NAME,
  3         SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE_AMOUNT
  4  FROM CUSTOMER C
  5  JOIN ORDERS O
  6  ON C.CUSTOMER_ID = O.CUSTOMER_ID
  7  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  8  HAVING SUM(O.TOTAL_AMOUNT) = (
  9      SELECT MAX(TOTAL_SPENDING)
 10      FROM (
 11          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING
 12          FROM ORDERS
```

5. Customers with fewer purchases.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP
BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) <=1 ORDER BY NUMBER_OF_ORDERS;

                    CUSTOMER_ID CUSTOMER_NAME           NUMBER_OF_ORDERS
------------------------------- ---------------------- ---------------
                            105 KAMESH                               1
                            102 VIJAY KUMAR                          1
                            101 SNEHA                                1
                            103 ANITHA                               1
                            104 SURESH                               1
```

4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending.

```
SQL> SELECT *
  2  FROM (
  3      SELECT C.CUSTOMER_ID,
  4             C.CUSTOMER_NAME,
  5             SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING
  6      FROM CUSTOMER C
  7      JOIN ORDERS O
  8      ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9      GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10      ORDER BY TOTAL_SPENDING DESC
```

2. Customers with maximum number of orders.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) = (SELECT MAX(ORDER_COUNT) FROM ( SELECT COUNT(*) AS ORDER_COUNT FROM ORDERS GROUP BY CUSTOMER_ID )) O
RDER BY CUSTOMER_NAME;

                    CUSTOMER_ID CUSTOMER_NAME          NUMBER_OF_ORDERS
--------------------------------- ---------------------- ---------------
                            103 ANITHA                             1
                            105 KAMESH                             1
                            101 SNEHA                              1
                            104 SURESH                             1
                            102 VIJAY KUMAR                        1
```

3. Frequent customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS FROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.
CUSTOMER_ID, C.CUSTOMER_NAME HAVING COUNT(O.ORDER_ID) >= 2 ORDER BY NUMBER_OF_ORDERS DESC;

no rows selected
```

4. High-value customers.

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS, SUM(O.TOTAL_AMOUNT) AS TOTAL_SPENDING FROM CUSTOMER C JOIN ORDERS O ON C.
CUSTOMER_ID = O.CUSTOMER_ID GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME HAVING SUM(O.TOTAL_AMOUNT) > 1000 ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_ID
----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
NUMBER_OF_ORDERS TOTAL_SPENDING
---------------- ---------------
       102
VIJAY KUMAR
               1            1250
```

6. Identify Best-Selling Products

1. Products with maximum sales quantity

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP
 BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
P03
CHICKEN FRIED RICE
                  5

P06
CHICKEN BURGER
                  4

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------

P02
BUTTER NAAN
                  3

P05
BROWNIE

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD
-------------------
                  2

P01
IDLI
                  2
```

2. Products generating highest revenue

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, SUM(OD.QUANTITY) AS TOTAL_QUANTITY_SOLD, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DE
TAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY TOTAL_QUANTITY_SOLD DESC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
P03
CHICKEN FRIED RICE
                  5          1250

P06
CHICKEN BURGER
                  4          1600

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------

P02
BUTTER NAAN
                  3           720

P05
BROWNIE

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
TOTAL_QUANTITY_SOLD TOTAL_REVENUE
------------------- -------------
                  2           600

P01
IDLI
                  2           500
```

3.Least-selling products.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_I
D GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY QUANTITY_SOLD ASC;

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
P01
IDLI
           2

P05
BROWNIE
           2

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------

P02
BUTTER NAAN
           3

P06
CHICKEN BURGER

PRODUCT
-------
PRODUCT_NAME
--------------------------------------------------------------------------------
QUANTITY_SOLD
-------------
           4

P03
CHICKEN FRIED RICE
           5
```

4.Products requiring promotion.

```
SQL> SELECT P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE, P.STOCK_QUANTITY, NVL(SUM(OD.QUANTITY), 0) AS QUANTITY_SOLD FROM PRODUCT P LEFT JOIN ORDER_DETAILS OD ON
P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME, P.PRICE, P.STOCK_QUANTITY HAVING NVL(SUM(OD.QUANTITY), 0) <= 1 AND P.STOCK_QUANTITY < 0
ORDER BY QUANTITY_SOLD, P.PRODUCT_NAME;

no rows selected
```

6.Perform Category-wise Sales Analysis

1.Total sales generated by each category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_SALES FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS O
D ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY TOTAL_SALES DESC;

CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
FAST FOOD
      2850

NORTH INDIAN
       720

DESSERTS
       600


CATEGORY_NAME
--------------------------------------------------------------------------------
TOTAL_SALES
-----------
SOUTH INDIAN
       500
```

1.Number of products sold in each category.

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY) AS PRODUCT_SOLD FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.PRODUCT_ID JOIN ORDER_DETAILS OD ON P.PROD
UCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY PRODUCT_SOLD DESC;

no rows selected
```

1. **Highest revenue-generating category.**

```
SQL> SELECT C.CATEGORY_NAME,
  2         SUM(OD.QUANTITY * OD.PRICE) AS TOTAL_REVENUE
  3  FROM CATEGORY C
  4  JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID
  5  JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID
  6  GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME
  7  HAVING SUM(OD.QUANTITY * OD.PRICE) = (
  8      SELECT MAX(CATEGORY_REVENUE)
  9      FROM (
 10          SELECT SUM(OD2.QUANTITY * OD2.PRICE) AS CATEGORY_REVENUE
 11          FROM CATEGORY C2
 12          JOIN PRODUCT P2 ON C2.CATEGORY_ID = P2.CATEGORY_ID
 13          JOIN ORDER_DETAILS OD2 ON P2.PRODUCT_ID = OD2.PRODUCT_ID
 14          GROUP BY C2.CATEGORY_ID
 15      )
 16  );

CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_REVENUE
-------------
FAST FOOD
         2850
```

2. **Average sales per category.**

```
SQL> SELECT C.CATEGORY_NAME, ROUND(AVG(OD.QUANTITY * OD.PRICE), 2) AS AVERAG
E_SALES FROM CATEGORY C JOIN PRODUCT P ON C.CATEGORY_ID = P.CATEGORY_ID JOIN
 ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY C.CATEGORY_ID, C.
CATEGORY_NAME ORDER BY AVERAGE_SALES DESC;

CATEGORY_NAME
--------------------------------------------------------------------------
----
AVERAGE_SALES
-------------
FAST FOOD
         1425

NORTH INDIAN
          720

DESSERTS
          600


CATEGORY_NAME
--------------------------------------------------------------------------
----
AVERAGE_SALES
-------------
SOUTH INDIAN
          500
```

7. **Generate Business Analytics Reports**

Report 1: Sales Performance Report

Highest order value

```
SQL> SELECT SUM(TOTAL_AMOUNT) AS TOTAL_SALES, COUNT(ORDER_ID) AS NUMBER_OF_O
RDERS, ROUND(AVG(TOTAL_AMOUNT), 2) AS AVERAGE_ORDER_VALUE, MAX(TOTAL_AMOUNT)
 AS HIGHEST_ORDER_VALUE FROM ORDERS;

TOTAL_SALES NUMBER_OF_ORDERS AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
----------- ---------------- ------------------- -------------------
       4070                5                 814                1250
```

Report 2: Customer Analytics Report

```
SQL> SELECT C.CUSTOMER_NAME, COUNT(O.ORDER_ID) AS NUMBER_OF_ORDERS, NVL(SUM(
O.TOTAL_AMOUNT), 0) AS TOTAL_SPENDING, COUNT(O.ORDER_ID) AS PURCHASE_FREQUEN
CY FROM CUSTOMER C LEFT JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID GROUP
 BY C.CUSTOMER_ID, C.CUSTOMER_NAME ORDER BY TOTAL_SPENDING DESC;

CUSTOMER_NAME
--------------------------------------------------------------------------------
----
NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
---------------- -------------- -------------------
VIJAY KUMAR
               1           1250                   1

KAMESH
               1            900                   1

SURESH
               1            890                   1


CUSTOMER_NAME
--------------------------------------------------------------------------------
----
NUMBER_OF_ORDERS TOTAL_SPENDING PURCHASE_FREQUENCY
---------------- -------------- -------------------
SNEHA
               1            550                   1

ANITHA
               1            480                   1
```

Report 3: Product Performance Report

```
SQL> SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS QUANTITY_SOLD, SUM(OD.QUANTI
TY * OD.PRICE) AS REVENUE_GENERATED FROM PRODUCT P JOIN ORDER_DETAILS OD ON
P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NAME ORDER BY
REVENUE_GENERATED DESC;

PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
CHICKEN BURGER
            4              1600

CHICKEN FRIED RICE
            5              1250

BUTTER NAAN
            3               720


PRODUCT_NAME
--------------------------------------------------------------------------------
----
QUANTITY_SOLD REVENUE_GENERATED
------------- -----------------
BROWNIE
            2               600

IDLI
            2               500
```

Report 4: Category Analysis Report

```
SQL> SELECT C.CATEGORY_NAME, SUM(OD.QUANTITY) AS TOTAL_PRODUCTS_SOLD, SUM(OD
.QUANTITY * OD.PRICE) AS TOTAL_REVENUE FROM CATEGORY C JOIN PRODUCT P ON C.C
ATEGORY_ID = P.CATEGORY_ID JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUC
T_ID GROUP BY C.CATEGORY_ID, C.CATEGORY_NAME ORDER BY TOTAL_REVENUE DESC;

CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------- -------------
FAST FOOD
                  9          2850

NORTH INDIAN
                  3           720

DESSERTS
                  2           600


CATEGORY_NAME
--------------------------------------------------------------------------------
----
TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
------------------- -------------
SOUTH INDIAN
                  2           500
```